Super-Spreaders, Firebreaks and Stolen Tokens: SIR Modelling of Self-Propagating Worms in the npm Dependency Network

Sunil J Varun Sithaarth

Department of Applied Mathematics and Computational Sciences
PSG College of Technology, Peelamedu, Coimbatore 641004, Tamil Nadu, India

November 10, 2026

Abstract

In September and November 2025 the self-propagating worms Shai-Hulud and Shai-Hulud 2.0 compromised roughly 203 and 795 npm packages. They did so by stealing maintainers' publish tokens during package installation and republishing those maintainers' packages with an injected install script. We model such worms as a two-channel SIR epidemic on the npm ecosystem.

- **Dependency channel.** A compromised dependency exposes its dependents, weighted by how permissive each SemVer range is and by whether consumers block lifecycle scripts.
- **Credential channel.** A leaked token lets the worm republish up to a capped number of the owner's packages.

Our main data set is a 10,000-package graph built from the npm registry. It is the dependency closure of the 4,686 most-depended-upon packages, cleaned of a reward-farming spam ring.

Calibration and validation. With one tick equal to one hour, we calibrate transmission, credential and patching rates to the observed plateau times and stolen-token share of both waves. Against the real compromised-package lists, victims share a maintainer with another victim far more often than random package sets do (98% vs. 49% for wave 1, 95% vs. 69% for wave 2). They are almost absent from the high-impact core.

Findings.

- With dependencies alone, spread is bounded: outbreaks started at the most-depended-upon packages reach only 2.6% of the core, and leaf-started outbreaks fizzle.
- With the calibrated credential channel, even a leaf patient zero reaches 19.6% (wave 1 parameters) to 55.4% (wave 2) of the core.
- Strict lockfiles and script blocking raise the dependency-channel epidemic threshold 12× and 6×. With stealable tokens, however, the threshold in the dependency transmission rate collapses to the bottom of our grid.
- Reactive patching prevents almost all spread if the worm is detected within an hour, but only 13% after a realistic 24-hour detection lag. Proactively immunizing the same number of high-degree packages still prevents 42%.

Credential hardening and proactive protection of central packages therefore matter more than dependency pinning.

1 Introduction

Modern JavaScript applications are assembled from hundreds of npm packages. Each package declares its own dependencies through SemVer ranges, and package managers resolve and install these transitively. This creates a dense, directed web of trust. A compromise of a single widely used package can reach a large part of the ecosystem [13, 23, 59]. The exposure is no longer hypothetical. Account takeovers and malicious maintainer handovers have repeatedly delivered malware through trusted packages [15, 19, 35, 36].

In 2025 the threat changed qualitatively, as worms began to propagate themselves:

- **Shai-Hulud** (September) ran a malicious script at install time. The script harvested npm publish tokens from developer machines and CI runners, then used each stolen token to republish up to 20 of the victim's own packages with the same payload injected [10, 24, 37].
- **Shai-Hulud 2.0** (November) used a preinstall hook and raised the per-token cap to 100 packages [44, 51].

Each newly infected package infected whoever installed it next. Spread was limited only by detection and registry intervention. These incidents raise network-scientific questions that static vulnerability reachability studies [12, 28, 58] cannot answer:

- How do the topology of the dependency graph and the ownership structure of maintainer accounts jointly determine how far and how fast such a worm spreads?
- Which ecosystem-level defences change the epidemic threshold?
- Under realistic detection delays, should a limited response capacity go to patching what is already infected, or to hardening what is not?

We address these questions with an SIR epidemic model of the npm ecosystem that has two contagion channels:

- a *dependency channel*, in which a compromised package exposes its dependents with a probability that depends on the dependents' SemVer constraints and on whether they execute lifecycle scripts;
- a *credential channel*, in which a compromised package's maintainer token is stolen and the worm republishes a capped number of that maintainer's packages.

We study three research questions:

RQ1 (origin topology). How much do outbreak size and speed depend on where the worm starts: a hub, a bridge, a mid-tier or a leaf package?

RQ2 (ecosystem regimes). How do strict lockfiles, disabled lifecycle scripts and trusted publishing shift the epidemic threshold?

RQ3 (interventions). At a fixed response budget, how does reactive patching compare with proactive immunization under realistic detection delays?

Contributions.

1. **Model.** We give, to our knowledge, the first epidemic-dynamics model of self-propagating npm worms that couples a SemVer-weighted dependency channel with a stolen-token credential channel (Section 4).

2. **Data.** We provide a cleaned 10,000-package npm dependency graph, built from the registry with provenance, install-script and maintainer metadata, plus a full-closure variant and a synthetic benchmark (Section 3). Cleaning removes a reward-farming spam ring that on its own would have dominated the spectral analysis.

3. **Calibration and validation.** We calibrate the model to the timelines of both Shai-Hulud waves and validate its channel structure against the real lists of compromised packages (Section 5).

4. **Evidence on defences.** We show that credential compromise, not dependency topology, governs the reach of a worm (RQ1). Dependency-side hardening raises the epidemic threshold only while tokens cannot be stolen (RQ2). Proactive immunization of high-degree packages outperforms reactive patching once detection takes between 12 and 24 hours or more (RQ3, Section 6). Null models, synthetic ensembles, sample sizes and parameter sweeps test the robustness of these findings (Section 7).

Code, data snapshots and the frozen experiment configuration are released with the paper.

2 Background and Related Work

Package dependency networks. Empirical studies of npm, PyPI, CRAN, RubyGems and Cargo describe dependency networks that grow super-linearly. In these networks a small number of packages accumulate very many transitive dependents, and trivial packages are pervasive [1, 13, 23]. The 2016 removal of `left-pad` showed how one tiny package can break a large part of the ecosystem [34]. Dependents constrain the versions they accept with SemVer ranges [43]. Caret (`^`) and tilde (`~`) ranges dominate in npm, so new releases propagate automatically [11, 41]. This permissiveness speeds up the uptake of security fixes [41], and it equally speeds up the uptake of malicious releases.

Supply-chain attacks on npm. Zimmermann et al. [59] showed that compromising a handful of maintainer accounts would let an attacker reach a large fraction of npm. Taxonomies of supply-chain attacks [25, 36] catalogue the main techniques:

- typosquatting [52];
- weak maintainer accounts [57];
- abuse of install-time lifecycle scripts [56].

Notable incidents include `event-stream` [35] and `ua-parser-js` [19]. In September 2025 the `chalk/debug` account takeover [15] and the s1ngularity/Nx compromise [3] preceded two waves of self-propagating malware.

The Shai-Hulud worms. Both waves followed the same pattern. A malicious package version ran a lifecycle script on the machine that installed it. The script scanned for credentials, exfiltrated them to public GitHub repositories, and used any npm token it found to republish the token owner's packages with the payload injected [10, 24, 37, 53].

- **Wave 1** (September 2025) capped republishing at 20 packages per token and compromised about 203 packages.
- **Wave 2** (November 2025) used a `preinstall` hook and a cap of 100, and compromised about 795 [44, 51].

Vendor analyses agree that the worm spread from maintainer to maintainer through installs, but generated most compromised packages by republishing with stolen tokens. As Pandya et al. [37] put it, the worm "cannot self-propagate without those credentials".

Defences. Three families of ecosystem defences are being deployed:

- **Lockfiles and pinning** freeze resolved versions. They delay the uptake of malicious and of benign releases alike [41, 58].
- **Blocking lifecycle scripts** removes the install-time execution that both waves relied on. Options include npm's `-ignore-scripts`, pnpm's default since version 10, and LavaMoat's allow-lists [26, 33, 42].
- **Provenance and trusted publishing** bind releases to CI identities, so a stolen long-lived token is no longer enough to publish [14, 18, 32, 50].

After Shai-Hulud, GitHub announced token-lifetime limits, revoked classic tokens and expanded trusted publishing [17, 45]. Compromised CI pipelines can still publish with valid provenance [3, 54], so these defences are partial.

Epidemics and immunization on networks. Epidemic models have been used for computer viruses since Kephart and White [21] placed them on directed sharing graphs. They were later applied to internet worms such as Code Red [60]. We build on four strands of theory:

- **SIR dynamics.** The model goes back to Kermack and McKendrick [22]; on networks, final outbreak sizes map to bond percolation [31].
- **Epidemic thresholds.** The threshold vanishes on scale-free networks with diverging degree moments [38, 40]. On general graphs it is governed by the spectral radius $\lambda_{\max}$ of the contagion matrix [6, 55]. On directed networks $1/\lambda_{\max}$ is only a lower bound, and directionality raises the true threshold [4, 27].
- **Immunization.** Targeting high-degree nodes is far more effective than random immunization [39]. Acquaintance immunization approximates targeting without global knowledge [9]. Betweenness-based removal fragments networks quickly [16, 20], although optimized strategies beat highest-betweenness heuristics [7, 48].
- **Methods.** We fit heavy tails with the method of Clauset et al. [8], as implemented by Alstott et al. [2]. Strictly scale-free real networks are rare [5].

Positioning. Prior work on propagation in package ecosystems is mostly about the *static reachability* of known vulnerabilities [12, 28, 30, 46, 58]. The closest dynamic model, by Schueller and Wachs [49], couples dependency and shared-maintainer failures in Rust, but it models non-adversarial maintainer exit. To our knowledge, no prior work models self-propagating npm worms as an epidemic process with a credential channel, calibrates such a model to observed incidents, or evaluates ecosystem defences against it.

3 Data

3.1 Registry crawl

We seed a breadth-first crawl of the npm registry with the 4,686 packages that have the most dependents, as listed in `npm-high-impact` (version 1.13.0). For every package the crawl fetches the latest-version manifest from `https://registry.npmjs.org` and extracts:

- the four dependency tables (production, optional, peer and development) with their raw version constraints;
- the presence of `preinstall`, `install` or `postinstall` scripts, or npm's `hasInstallScript` flag;
- build provenance, from the `dist.attestations` field;
- the list of maintainer accounts.

The crawl follows installed dependency types (production, optional, peer) up to a node budget. It does not follow development dependencies, which consumers do not install. Ecosystem-wide dependent counts come from the deps.dev API.

We classify every constraint by how permissive it is to automatic updates, following node-semver semantics:

- *exact* (`1.2.3`), *tilde* (`~1.2.3`), *caret* (`^1.2.3`) or *wildcard* (`*`, dist-tags, open upper bounds);
- caret ranges on `0.x` behave like tilde, and `^0.0.x` like exact;
- for `||` unions the loosest branch wins;
- `npm:` aliases resolve to their target;
- git, file and URL specifiers are *non-registry* and are dropped, since registry publishes cannot reach them.

We build three graphs from the same seed list:

- a 5,000-package sample;
- the 10,000-package *main* graph;
- the *full closure*, in which the crawl ran out of new packages at 14,660 packages.

Edges to packages outside a sample are dropped.

Table 1: The graphs used in this study. Blast radius is the number of transitive dependents. The synthetic graph is a DAG by construction, so its $\lambda_{\max} = 0$ and $\tau_c = \infty$.

Graph	Packages	Edges	Largest SCC	$\lambda_{\max}(W)$	τ_c	Max. blast radius
npm 5k	5,000	15,907	5	2.00	0.500	761
npm 10k (main)	10,000	43,233	22	4.92	0.203	1,456
npm full closure	14,624	60,041	46	8.98	0.111	2,305
Synthetic	2,000	3,927	1	0.00	∞	1,999

Spam-ring cleaning. The full closure contained a complete 25-package clique of mutually dependent packages with fruit-pair names, all owned by one maintainer. It belongs to a farm of 36 reward-farming packages that entered the crawl through a single dependent. On its own this clique set $\lambda_{\max} = 24$. We therefore drop every strongly connected component with at least 10 packages and edge density at least 0.5, together with the other packages of maintainers who own most of such a ring. Legitimate monorepo cycles survive this filter because they are far sparser, for example `@stdlib` (density 0.26), `@yarnpkg` (0.13) and `@eggjs` (0.10).

3.2 Synthetic benchmark

As a controlled baseline we generate directed scale-free graphs with the Bollobás–Borgs–Chayes–Riordan model, using `networkx`'s `scale_free_graph` with $n = 2{,}000$ and a narrowed out-degree. Because node index equals creation order, we orient every edge from newer to older packages. This removes a large artificial cycle among hubs that the raw generator creates, which real npm does not have. SemVer types, install scripts, provenance and maintainers are assigned at empirical rates. Maintainer portfolios follow a Zipf-like distribution.

3.3 Structure of the main graph

Table 1 summarises the graphs. The main graph has 10,000 packages and 43,233 dependency edges. Its SemVer mix is 71.2% caret, 15.0% exact, 7.6% tilde and 6.2% wildcard. Only 1.0% of packages declare install scripts, while 21.5% publish with provenance.

The graph is close to a DAG. It has 47 non-trivial strongly connected components, the largest of which has 22 packages. The spectral radius of the weighted contagion matrix is $\lambda_{\max} = 4.92$, which sets the mean-field threshold $\tau_c = 1/\lambda_{\max} = 0.203$. This value is determined by a single dense `@stdlib` cluster rather than by the ecosystem as a whole.

The maximum blast radius, i.e. the largest number of transitive dependents of any package, is 1,456, and the mean is 45.0. Out-degree (direct dependents) is heavy-tailed (Figure 1). A discrete power-law fit gives $\alpha = 2.08$ for $k \geq 5$, but a likelihood-ratio test prefers a lognormal (normalized log-likelihood ratio -2.74, $p = 0.006$). This agrees with the finding that strictly scale-free real networks are rare [5]. Degree assortativity is weakly negative ($r_{\text{out,in}} = -0.069$).

4 Model

4.1 Contagion graph

Let $G_{\text{dep}} = (V, E)$ contain an edge $u \to v$ whenever package u depends on v. Malicious code flows against this arrow: a compromised v exposes every u that installs it. Our contagion graph is therefore the reversal $G = G_{\text{dep}}^{\top}$, with edges $v \to u$. Each node is susceptible (S), infected (I)

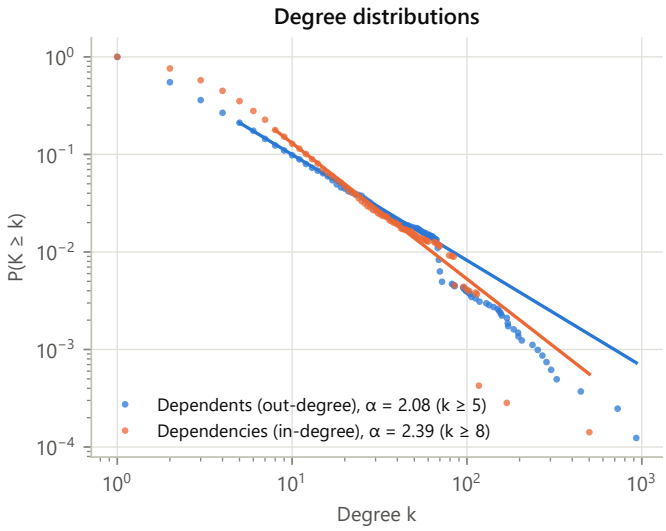


Figure 1: Complementary cumulative degree distributions of the main graph, with discrete power-law fits above $x_{\min}$. Dependents correspond to the contagion out-degree.

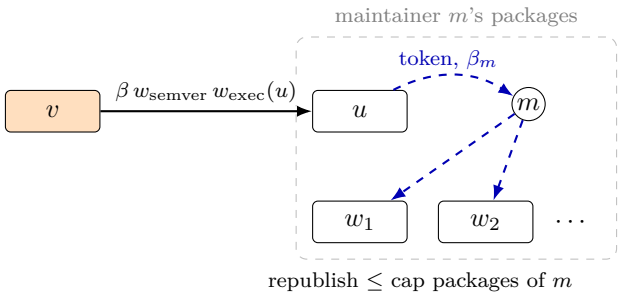


Figure 2: The two contagion channels. In the dependency channel (solid), compromised v infects dependent u when u's maintainer installs it and the injected install script runs. In the credential channel (dashed), an infected package leaks its maintainer's token, and the worm republishes up to a capped number of that maintainer's other packages.

or recovered (R). Here *infected* means that the package has published a malicious version, and *recovered* means that it has been patched, unpublished or protected. Figure 2 shows both contagion channels.

4.2 Dependency channel

In each discrete tick, every infected v infects each susceptible successor u independently with probability

$$\beta_{vu} = \min(1,\ \beta \cdot w_{\text{semver}}(v,u) \cdot w_{\text{exec}}(u) \cdot w_{\text{prov}}(u)). \tag{1}$$

The three weights are:

- **SemVer weight.** w_{semver} encodes how readily u's constraint accepts an automatic malicious release: exact 0.05, tilde 0.85, caret and wildcard 1.0.
- **Script execution.** The worm injects its own lifecycle script into every package it republishes, so every infected source carries the payload. What varies is the consumer: $w_{\text{exec}}(u) = 0.1$

if u's maintainers or CI block lifecycle scripts, and 1 otherwise. The residual 0.1 covers runtime execution paths. In the status quo we assume 10% of packages block scripts, reflecting pnpm ≥10, Bun and `--ignore-scripts` users. The blocking sets are nested across regimes. The original static formulation, which boosts packages that *declare* install scripts, made script blocking ineffective by construction; we keep it only as a variant.

- **Provenance.** $w_{\text{prov}}(u)$ is a provenance factor applied in hardened regimes (Section 6.2).

4.3 Credential channel

In each tick, every infected package leaks each of its not-yet-compromised maintainers' tokens with probability β_m. A compromised maintainer has up to c of their packages republished in the next tick, chosen at random; packages with provenance are republished with probability w_{prov}. The cap c mirrors the observed limits of 20 (wave 1) and 100 (wave 2) packages per token.

4.4 Recovery, interventions and metrics

Recovery. Infected packages recover with probability γ per tick from the *detection tick* onward. Nobody patches a worm that has not yet been discovered. One tick corresponds to one hour.

Interventions. An intervention policy moves up to B nodes per tick to R from the detection tick onward. It can target infected nodes uniformly at random (*random patching*) or in order of contagion out-degree (*hub patching*). Alternatively, a set of K nodes is immunized at $t = 0$, in one of three ways:

- *firebreak*: highest betweenness;
- *hub*: highest out-degree;
- *acquaintance*: random neighbours of random nodes [9].

Metrics. The final outbreak size R_∞ is the fraction of packages *ever* infected. It excludes immunized and patched nodes, which would otherwise inflate $|R(\infty)|$ precisely when defences work. We also report:

- T_{80}: the time to infect 80% of the patient zero's dependency-reachable set;
- T_{90}: the time to reach 90% of a run's final size, i.e. the plateau;
- peak prevalence;
- infection-tree depth;
- the share of infections caused by the credential channel;
- the probability of a major outbreak ($R_\infty \geq 5\%$).

Table 2: Parameters. Ticks are hours. SH1 and SH2 values are calibrated in Section 5. Patching starts at the detection tick.

Symbol	Meaning	Spec reference	SH1-calibrated	SH2-calibrated
β	dependency transmission	0.15	0.02	0.20
β_m	token leak per maintainer and tick	0	0.02	0.10
γ	patching rate after detection	0.05	0.10	0.05
c	packages republished per token	20	20	100
t_{det}	detection tick (h)	0	24	0

4.5 Simulation and statistics

Each configuration is run 100 times (50 for the RQ2 grids). Run i of a configuration uses child i of a NumPy `SeedSequence`. Configurations that share a seed are therefore *paired*: run i gets the same patient-zero draw and random stream across strategies and regimes. Outbreak sizes are bimodal, so we report percentile-bootstrap 95% confidence intervals and give the probability of a major outbreak separately from its size. Paired reductions use a paired bootstrap.

The engine is vectorised over a CSR representation of G, so 100 runs on the 10k graph take about one second. We cross-validated it against NDlib's SIR model [47]: with homogeneous weights, mean outbreak sizes agree within sampling error.

5 Calibration and Empirical Validation

5.1 Incident data

We compiled the packages compromised in both Shai-Hulud waves from public indicator lists and vendor reports [24, 37, 44, 51]. This gives 203 packages for wave 1 and 795 for wave 2. Publish timestamps come from registry metadata, which survives takedown. Three facts from the incidents anchor the calibration:

1. **Detection lag.** Wave 1 went about 24 h from the first malicious publish to the first public signal. Wave 2 was flagged by automated feeds within minutes.

2. **Plateau.** 90% of the compromised versions had appeared 37 h (wave 1) and 13.4 h (wave 2) after the first publish.

3. **Mechanism.** Most compromised packages were produced by stolen-token republishing. Publish bursts match the per-token caps, and only about 230 of the about 800 wave-2 packages have any dependents at all. We therefore target a credential-channel share of 75%.

5.2 Calibration

We grid-search $\beta \in [0.005, 0.3]$, $\beta_m \in [0.005, 0.5]$ and $\gamma \in [0.05, 0.4]$ on the main graph, with each wave's cap and detection tick fixed. Each configuration gets 40 runs, each from a uniformly random patient zero. Plateau time and channel share are evaluated on major outbreaks only, since the observed waves are by definition outbreaks that took off. The loss is the squared log-error of the median T_{90} plus the squared, scaled error of the channel share.

Both best fits lie inside the grid:

Table 3: Clustering of real Shai-Hulud victims. A package counts as *co-maintained* if it shares a maintainer with another victim, and as *dependency-linked* if it has an installed dependency edge to another victim. The null is the mean over 200 random sets of equal size from the full closure. The last column counts victims present in the full-closure graph.

		Co-maintained		Dependency-linked		In core
Wave	n	observed	null	observed	null	graph
Shai-Hulud 1	189	98%	49%	32%	5%	2
Shai-Hulud 2	732	95%	69%	23%	14%	7

- **SH1-calibrated:** $\beta = 0.02$, $\beta_m = 0.02$, $\gamma = 0.10$, giving a plateau at $T_{90} = 41.5\,\text{h}$ with 76% of infections via the credential channel.
- **SH2-calibrated:** $\beta = 0.20$, $\beta_m = 0.10$, $\gamma = 0.05$, giving $T_{90} = 14.0\,\text{h}$ and a credential share of 77%.

The data constrain transmission well. Configurations within a loss tolerance of 0.1 of the optimum span $\beta \in [0.010, 0.050]$ and $\beta_m \in [0.005, 0.100]$ for wave 1, and $\beta \in [0.100, 0.300]$ and $\beta_m \in [0.020, 0.200]$ for wave 2. They do not identify γ, since every value we tried fits, nor the absolute outbreak size, since median major-outbreak sizes range over 65–7,192 packages for wave 1.

We therefore treat outbreak sizes in the high-impact core as *projections*. They answer the question "what if a worm with Shai-Hulud's dynamics reached the core?", not "what did Shai-Hulud do".

5.3 Validation of the channel structure

If the credential channel dominates, victims should cluster by shared maintainer much more than by dependency links. We fetched current registry metadata, never package contents, for 189 of the wave-1 and 732 of the wave-2 packages. We then compared them with random sets of equal size drawn from the full closure (Table 3).

The pattern is clear:

- 98% of wave-1 victims share a maintainer with another victim, against 49% in the null. For wave 2 the figures are 95% and 69%.
- Installed dependency links are much rarer: 32% and 23% of victims.
- Almost no victims belong to the high-impact core: 2 wave-1 and 7 wave-2 packages appear in the full closure.

The null is drawn from the high-impact closure, where monorepo organisations inflate co-maintenance, so the contrast is conservative. Taken together, the evidence shows that the real waves spread through stolen tokens among the periphery's small maintainers. Our experiments ask what happens when such a worm enters the core.

6 Results

All results in this section use the main graph and the script-blocking status quo, unless stated otherwise.

Table 4: RQ1: mean final outbreak size R_∞, in parentheses the probability of a major outbreak ($R_\infty \geq 5\%$), and below each value its 95% bootstrap CI, by patient-zero group. Each cell aggregates 1,000 runs.

Scenario	Hub	Bridge	Mid-tier	Leaf
Spec reference	5.3% (46%)	0.8% (0%)	0.2% (0%)	0.0% (0%)
	[5.1%, 5.4%]	[0.8%, 0.9%]	[0.2%, 0.2%]	[0.0%, 0.0%]
Dependency only	2.6% (11%)	0.2% (0%)	0.0% (0%)	0.0% (0%)
	[2.5%, 2.7%]	[0.2%, 0.3%]	[0.0%, 0.0%]	[0.0%, 0.0%]
SH1-calibrated	55.8% (100%)	48.7% (95%)	31.8% (63%)	19.6% (40%)
	[55.6%, 55.9%]	[48.0%, 49.4%]	[30.2%, 33.3%]	[18.1%, 21.0%]
SH2-calibrated	85.8% (100%)	84.2% (98%)	78.8% (92%)	55.4% (64%)
	[85.7%, 85.8%]	[83.4%, 84.9%]	[77.2%, 80.2%]	[52.8%, 57.8%]

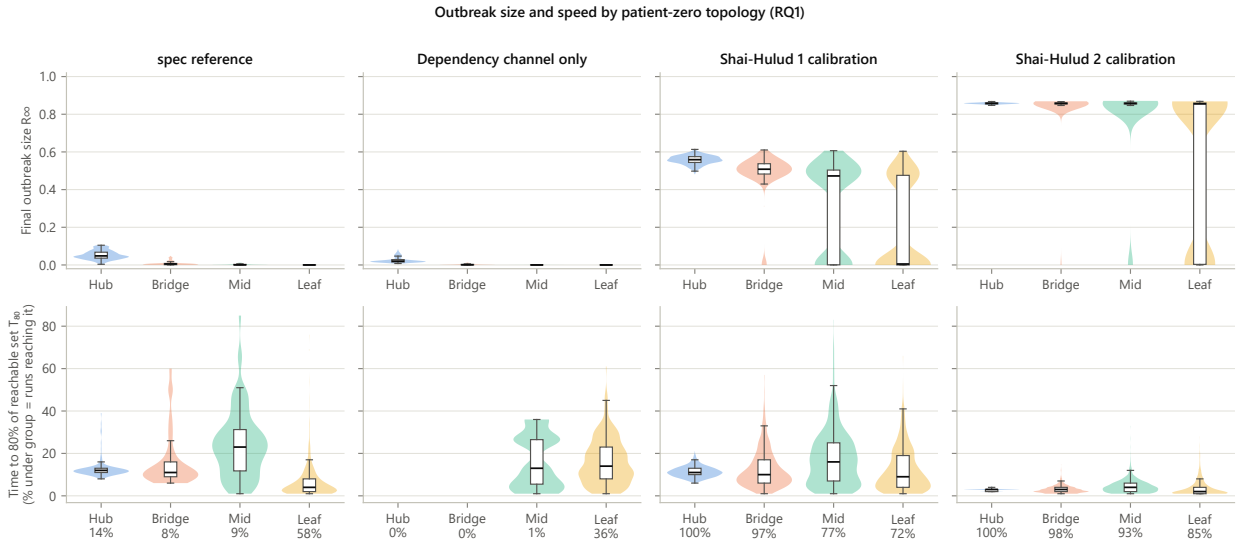


Figure 3: RQ1. Top: distributions of the final outbreak size R_∞ by origin group and scenario. Bottom: time to infect 80% of the dependency-reachable set, for runs that reach it (the share of runs is shown under each group).

6.1 RQ1: Does the origin of an outbreak matter?

We use four origin groups, each disjoint from the others:

- *hubs*: the ten packages with the most direct dependents;
- *bridges*: the ten with the highest betweenness among the remaining packages;
- *mid-tier*: ten packages at the median k-core;
- *leaves*: ten packages in the bottom 20% by contagion out-degree.

We start 100 outbreaks from each package in each group, under four parameter scenarios (Table 4 and Figure 3).

Dependency-only spread is bounded and hub-driven. In the spec-reference and dependency-only scenarios, outbreaks seeded at hubs reach 5.3% and 2.6% of the core, respectively. Outbreaks seeded at leaves essentially never spread. Because the graph is nearly acyclic, an outbreak can only

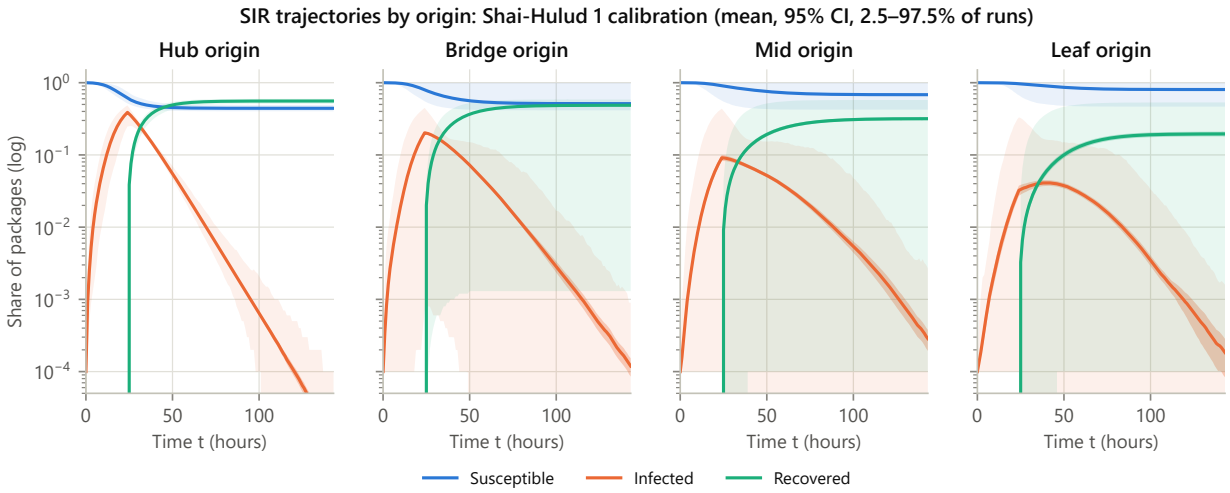


Figure 4: S/I/R trajectories (log scale) under SH1-calibrated parameters, by origin group: mean, 95% CI of the mean, and the 2.5–97.5% envelope of runs. Patching starts at $t = 24$ h.

travel down the dependents of its origin, and transmission is stochastic with $\beta < 1$. A hub therefore reaches only a fraction of its static blast radius before patching stops it. Topology matters strongly here, and the super-spreader hypothesis holds.

The credential channel erases most of the origin effect. With the SH1-calibrated credential channel:

- hub-seeded outbreaks reach 55.8% of the core;
- outbreaks seeded at leaves, which cannot spread at all through dependencies, still reach 19.6% on average, with a major outbreak in 40% of runs.

Under SH2 parameters the corresponding figures are 85.8% and 55.4%. Between 70% and 76% of infections come through stolen tokens. Outbreak-size distributions become strongly bimodal (Figure 3): an outbreak either dies out before a token leaks or takes over most of the core. Origin topology mainly changes the *probability* of take-off, not the size of outbreaks that do take off. Figure 4 shows the resulting epidemic curves.

6.2 RQ2: Which ecosystem regimes shift the threshold?

We compare four regimes:

- *status quo*: the observed SemVer mix, with 10% of packages blocking scripts;
- *strict lockfile*: loose constraints pinned until 80% of edges are exact;
- *scripts disabled*: 70% of packages block lifecycle scripts;
- *hardened*: strict lockfiles, scripts disabled and trusted publishing on the top 5% of hubs. In the hardened regime, stolen-token publishes to these hubs succeed with probability 0.1.

For each regime we sweep a 13×5 grid of (β, γ) with hub-seeded outbreaks and estimate the critical β_c at which the mean R_∞ reaches 5% (Table 5 and Figure 5).

Table 5: RQ2, dependency channel only ($\beta_m = 0$): empirical critical transmission rate β_c at which mean R_∞ reaches 5%. "none" means that no grid value up to $\beta = 1$ produced a major outbreak.

Regime	$\gamma = 0.02$	$\gamma = 0.05$	$\gamma = 0.10$	$\gamma = 0.20$	$\gamma = 0.40$
Status quo	0.045	0.125	0.274	0.363	0.499
Strict lockfile	0.514	0.917	none	none	none
Scripts disabled	0.279	0.642	none	none	none
Hardened	none	none	none	none	none

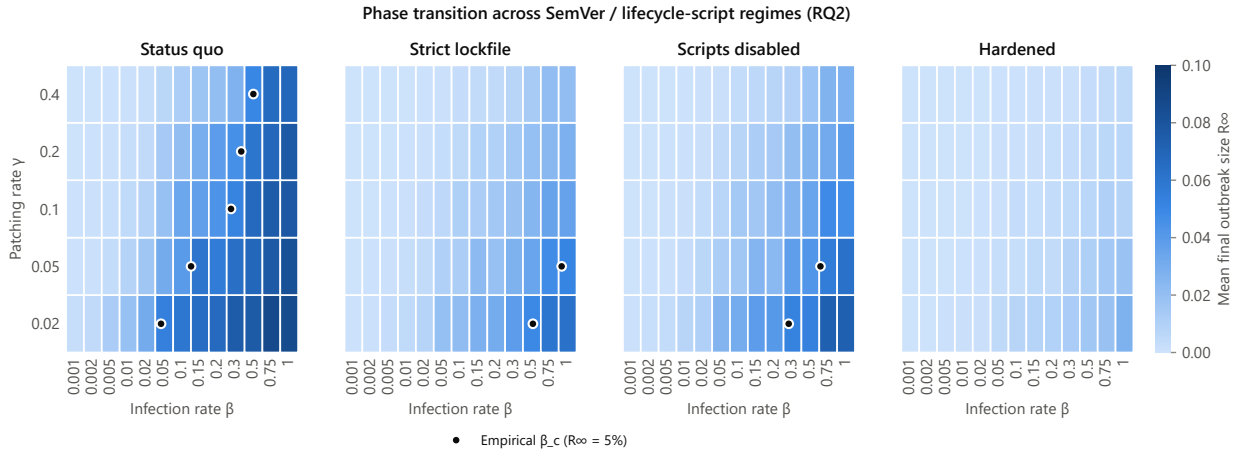


Figure 5: RQ2: mean R_∞ over the (β, γ) grid for each regime, dependency channel only. Markers show the empirical β_c.

Without stolen tokens, dependency-side hardening works. At $\gamma = 0.02$ the status quo has $\beta_c = 0.045$. Strict lockfiles raise it to 0.514 (about 12×), and blocking scripts on 70% of packages raises it to 0.279 (about 6×). The hardened regime never produces a major outbreak. The spectral prediction $\tau_c = 1/\lambda_{\max} = 0.203$ lies far from these empirical thresholds. This is expected for SIR on a nearly acyclic directed graph, where $1/\lambda_{\max}$ is only a lower bound that is set by small monorepo cycles [27].

With stolen tokens, the dependency threshold collapses. With the calibrated credential rates $\beta_m \in \{0.02, 0.10\}$, the estimated β_c falls to the bottom of our grid (0.001) in 75% of regime–γ combinations. The credential channel alone sustains the epidemic, so the relevant threshold is in β_m, not β. Table 6 reports the critical β_m per regime at the SH1 transmission rate. The critical token-leak rate rises with the patching rate, and hardening raises it about 7× at $\gamma = 0.1$ (from 0.002 to 0.015). The SH2 rate ($\beta_m = 0.10$) exceeds the threshold of every regime at every patching rate we tried. The SH1 rate ($\beta_m = 0.02$) is above threshold in the status quo except at the fastest patching rate, where it sits at the threshold ($\beta_{m,c} = 0.020$ at $\gamma = 0.4$). It falls below threshold only with fast patching: at $\gamma = 0.2$ only in the hardened regime, and at $\gamma = 0.4$ also with strict lockfiles (0.031) or disabled scripts (0.028). Dependency-side defences thus shift the credential threshold too, but not by enough to stop a worm that leaks tokens as readily as the real ones did.

Table 6: RQ2b: empirical critical token-leak rate $\beta_{m,c}$ per regime at $\beta = 0.02$.

Regime	$\gamma = 0.02$	$\gamma = 0.05$	$\gamma = 0.10$	$\gamma = 0.20$	$\gamma = 0.40$
Status quo	$\leq$0.001	0.001	0.002	0.008	0.020
Strict lockfile	0.001	0.002	0.005	0.015	0.031
Scripts disabled	$\leq$0.001	0.001	0.004	0.012	0.028
Hardened	0.002	0.006	0.015	0.038	0.087

Table 7: RQ3: share of the no-intervention outbreak prevented (paired, mean over 100 runs) at $B = 50$ per hour, by detection delay. The last row gives the no-intervention R_∞, which grows with the delay because natural patching also waits for detection.

Strategy	1 h	6 h	12 h	24 h	48 h
Hub patching	100%	90%	52%	13%	3%
Random patching	100%	76%	23%	5%	1%
Firebreak (betweenness)	3%	13%	27%	29%	26%
Hub immunization	6%	23%	38%	42%	40%
Acquaintance	3%	17%	32%	35%	34%
No intervention R_∞	47%	48%	49%	55%	68%

6.3 RQ3: Patch reactively or immunize proactively?

Outbreaks are seeded at hubs under SH1-calibrated parameters. Both reactive patching and natural recovery begin only at the detection delay $t_{\text{det}} \in \{1, 6, 12, 24, 48\}$ h.

- Reactive policies treat up to B packages per hour, either hubs first or at random.
- Proactive policies immunize K packages at $t = 0$, choosing them by betweenness, by degree or by acquaintance sampling.

To compare at equal cost, K is set to the mean number of packages that hub patching treats at the same B and delay. Table 7 reports paired reductions in R_∞ at $B = 50$ packages per hour (0.5% of the graph), and Figure 6 shows all budgets.

Reactive patching works only with fast detection. Detected within an hour, the outbreak is almost fully contained by either reactive policy (hub patching prevents 100%). Its value drops quickly with delay: 90% prevented at 6 h, 52% at 12 h, and only 13% at 24 h, which was the lag of wave 1.

Proactive immunization holds up. Immunizing high-degree packages at the same total cost prevents 38% at 12 h and 42% at 24 h. The two approaches cross over between 12 and 24 h of delay: reactive patching is still ahead at 12 h, and proactive immunization is well ahead at 24 h and 48 h. Acquaintance immunization, which needs no global knowledge, comes close (35% at 24 h).

Degree beats betweenness at realistic budgets. At $B = 50$ per hour, matching costs means immunizing $K = 1{,}253$ packages (12.5% of the graph). At this budget betweenness "firebreaks" prevent only 29% at 24 h. Degree-based immunization wins at every budget up to 1.0% of packages

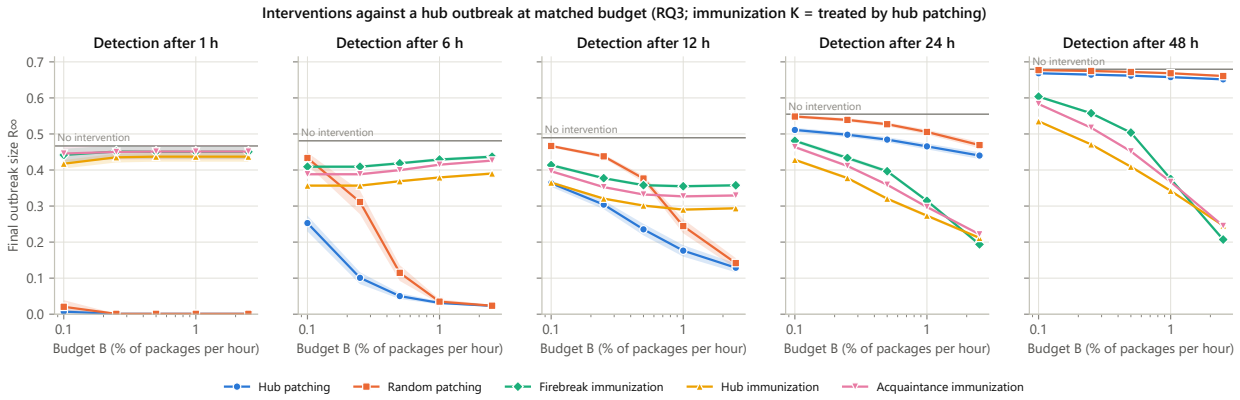


Figure 6: RQ3: final outbreak size against the intervention budget B (95% CI), by detection delay. Proactive strategies immunize the same total number of packages that hub patching treats.

per hour. Only at the largest budget we tested (2.5% per hour, $K = 2{,}649$) does betweenness edge ahead (65% vs. 62%). A plausible explanation is that betweenness targets packages that cut dependency paths, whereas under the credential channel most infections bypass those paths through stolen tokens; we have not tested this mechanism directly. Betweenness is a heuristic; optimized immunization sets could do better [48]. Our claim is therefore limited to the comparison with degree-based and random strategies at matched cost.

7 Robustness

Synthetic ensemble. Across 6 synthetic scale-free graphs (DAGs with $n = 2{,}000$) under SH1 parameters, hub-seeded outbreaks reach $38.3\% \pm 5.0\%$ and leaf-seeded ones $5.4\% \pm 2.8\%$ (mean $\pm$ s.d. across seeds). Hub immunization is the best RQ3 strategy at 24 h delay and maximum budget in 6/6 graphs. The qualitative RQ1 and RQ3 conclusions therefore do not depend on the particular structure of the npm crawl. The synthetic graphs differ from npm in the size of the credential effect: leaves reach much less, because synthetic maintainer portfolios are smaller and assigned independently of the topology.

Sample size and null models. Table 8 repeats the key quantities on three graph variants:

- the 5k sample;
- the full 14,624-package closure;
- five degree-preserving rewirings of the main graph [29]. These keep every package's in- and out-degree and the SemVer mix, but randomise who depends on whom.

The rewirings keep maintainer ownership fixed, so they separate the role of dependency structure from that of the credential channel.

Three patterns stand out in Table 8:

- **npm's wiring dampens spread.** The 5 rewired graphs produce *larger* outbreaks than the real one. Dependency-only hub outbreaks reach 7.6%–8.1% instead of 2.6%, and SH1 leaf outbreaks reach 32%–43% instead of 19.6%. Random rewiring destroys npm's layered, nearly acyclic structure and creates cycles, so the real topology is protective relative to its degree sequence alone.

Table 8: Robustness of the key quantities across graph variants. R_∞ values are means over 1,000 runs per origin group.

Graph	Hub R_∞ (dep. only)	Hub R_∞ (SH1)	Leaf R_∞ (SH1)	Best RQ3 strategy (24 h, max. budget)
npm 5k	2.3%	55.7%	22.4%	Hub immunization
npm 10k (main)	2.6%	55.8%	19.6%	Firebreak (betweenness)
npm full closure	2.2%	51.7%	7.8%	Firebreak (betweenness)
null model 0	8.1%	64.2%	31.8%	Acquaintance
null model 1	7.6%	64.4%	42.8%	Acquaintance
null model 2	8.0%	64.3%	35.0%	Hub immunization
null model 3	7.6%	64.3%	33.0%	Acquaintance
null model 4	8.1%	64.4%	38.3%	Acquaintance

- **The leaf effect is a share of the graph.** SH1 leaf outbreaks reach 22.4%, 19.6% and 7.8% of the 5k, 10k and full-closure graphs. In absolute terms that is between roughly 1,100 and 2,000 packages, so the share does not scale simply with graph size. We therefore report the leaf effect as a range rather than a single figure.
- **The RQ3 ranking is stable.** Proactive immunization beats reactive patching at a 24 h delay on every variant. Which proactive heuristic is best at the largest budget varies between graphs (degree, betweenness or acquaintance), consistent with the convergence of the strategies at large K in Section 6.3.

Parameter sensitivity. Around the SH1 calibration (50 runs per setting) we vary $\beta \in \{0.01, 0.02, 0.05\}$ and $\gamma \in \{0.05, 0.1, 0.2\}$. The conclusions hold across all of them:

- a proactive strategy is the best RQ3 response at 24 h delay and the largest budget in 9/9 settings;
- at $B = 50$ per hour, hub immunization beats hub patching in 9/9 settings and betweenness immunization in 9/9;
- the hub-to-leaf ratio of R_∞ stays between 2.0 and 5.0, so leaf-started outbreaks remain substantial whenever tokens can be stolen.

8 Discussion

Implications for defenders. Our results point to a clear order of priorities.

1. **Credentials first.** The credential channel turns a bounded, hub-dependent process into one that can take over the core from almost any origin (RQ1). It also removes the epidemic threshold that dependency-side defences create (RQ2). Measures that break this channel therefore address the mechanism that actually drove both Shai-Hulud waves: short-lived and scoped tokens, trusted publishing that makes stolen tokens useless, and caps or rate limits on publishing from a single credential.
2. **Then dependency hardening.** Lockfiles and blocking lifecycle scripts are still valuable. They raise the dependency-channel threshold by roughly an order of magnitude, and script blocking directly stops the install-time execution that harvests tokens. However, they are not sufficient while tokens remain stealable.

3. **Proactive protection over reactive patching.** With limited response capacity and detection lags of a day, protecting the most-depended-upon packages in advance beats chasing infected ones. At realistic budgets degree is a better target than betweenness; at very large budgets the two converge.

Lockfiles have a known cost: they slow the uptake of security fixes [41]. And as the s1ngularity and Mini Shai-Hulud incidents show, trusted publishing does not protect against a compromised CI pipeline [3, 54].

Limitations.

- **Scope of the graph.** Our graph is the high-impact core, not all of npm. The real waves mostly struck peripheral packages (Section 5), so absolute outbreak sizes in the core are projections. Calibration identifies dynamics (timescales and channel shares), not size.
- **Upper-bound assumptions.** The credential channel assumes that every infected package can leak its maintainers' tokens. Real leakage depends on whether a token was present on the infected machine, so our β_m absorbs this probability.
- **Current metadata.** Maintainer lists and dependencies were fetched in 2026, not as they were at the time of the incidents.
- **Assumed adoption shares.** The shares of packages that block scripts (10% and 70%) are assumptions; we found no reliable measurement of them.
- **Static immunization.** Our immunization sets are static. Recalculated betweenness or optimized sets could perform better [20, 48].
- **Remaining data quality.** A 17-package, low-density cluster of oddly named packages remains in the full closure below our spam threshold.

Ethics. We never downloaded, installed or executed compromised packages. Validation uses registry metadata and public indicator lists only.

9 Conclusion

We modelled self-propagating npm worms as a two-channel SIR epidemic and calibrated it to the two Shai-Hulud waves. We validated its structure against the real victims, who cluster by shared maintainer rather than by dependency links. We then used the model to evaluate where outbreaks start, which ecosystem regimes change the epidemic threshold, and how to spend a limited response budget.

Dependency topology alone confines outbreaks to the dependents of a few hubs, and dependency-side hardening raises that bound further. Stolen publish tokens remove it: with Shai-Hulud-like dynamics, a worm that enters the high-impact core from an obscure leaf package can reach a large part of it, and reactive patching after a day's delay barely helps. Breaking the credential channel, and proactively protecting the most-depended-upon packages, are the most effective measures against the next worm.

References

[1] Rabe Abdalkareem, Olivier Nourry, Sultan Wehaibi, Suhaib Mujahid, and Emad Shihab. Why do developers use trivial packages? an empirical case study on npm. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 385–395. ACM, 2017. doi: 10.1145/3106237.3106267.

[2] Jeff Alstott, Ed Bullmore, and Dietmar Plenz. powerlaw: A python package for analysis of heavy-tailed distributions. *PLoS ONE*, 9(1):e85777, 2014. doi: 10.1371/journal.pone.0085777.

[3] Merav Bar and Rami McCarthy. s1ngularity: supply chain attack leaks secrets on GitHub: everything you need to know. Wiz Blog, August 2025. URL `https://www.wiz.io/blog/s1ngularity-supply-chain-attack`. Published 27 August 2025, updated 29 August 2025.

[4] Marián Boguñá and M. Ángeles Serrano. Generalized percolation in random directed networks. *Physical Review E*, 72(1):016106, 2005. doi: 10.1103/PhysRevE.72.016106.

[5] Anna D. Broido and Aaron Clauset. Scale-free networks are rare. *Nature Communications*, 10: 1017, 2019. doi: 10.1038/s41467-019-08746-5.

[6] Deepayan Chakrabarti, Yang Wang, Chenxi Wang, Jurij Leskovec, and Christos Faloutsos. Epidemic thresholds in real networks. *ACM Transactions on Information and System Security*, 10(4):1–26, 2008. doi: 10.1145/1284680.1284681.

[7] Yiping Chen, Gerald Paul, Shlomo Havlin, Fredrik Liljeros, and H. Eugene Stanley. Finding a better immunization strategy. *Physical Review Letters*, 101(5):058701, 2008. doi: 10.1103/PhysRevLett.101.058701.

[8] Aaron Clauset, Cosma Rohilla Shalizi, and M. E. J. Newman. Power-law distributions in empirical data. *SIAM Review*, 51(4):661–703, 2009. doi: 10.1137/070710111.

[9] Reuven Cohen, Shlomo Havlin, and Daniel ben Avraham. Efficient immunization strategies for computer networks and populations. *Physical Review Letters*, 91(24):247901, 2003. doi: 10.1103/PhysRevLett.91.247901.

[10] Cybersecurity and Infrastructure Security Agency. Widespread supply chain compromise impacting npm ecosystem. CISA Alert, September 2025. URL `https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem`. Released 23 September 2025.

[11] Alexandre Decan and Tom Mens. What do package dependencies tell us about semantic versioning? *IEEE Transactions on Software Engineering*, 47(6):1226–1240, 2021. doi: 10.1109/TSE.2019.2918315.

[12] Alexandre Decan, Tom Mens, and Eleni Constantinou. On the impact of security vulnerabilities in the npm package dependency network. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR)*, pages 181–191. ACM, 2018. doi: 10.1145/3196398.3196401.

[13] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24 (1):381–416, 2019. doi: 10.1007/s10664-017-9589-y.

[14] Brian DeHamer and Philip Harrison. Introducing npm package provenance. The GitHub Blog, April 2023. URL https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/. Published 19 April 2023.

[15] Charlie Eriksen. npm debug and chalk packages compromised. Aikido Security Blog, September 2025. URL https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised. Published 8 September 2025.

[16] Linton C. Freeman. A set of measures of centrality based on betweenness. *Sociometry*, 40(1): 35–41, 1977. doi: 10.2307/3033543.

[17] GitHub. npm classic tokens revoked, session-based auth and CLI token management now available. GitHub Changelog, December 2025. URL https://github.blog/changelog/2025-12-09-npm-classic-tokens-revoked-session-based-auth-and-cli-token-management-now-available/. Published 9 December 2025.

[18] GitHub. npm trusted publishing with OIDC is generally available. GitHub Changelog, July 2025. URL https://github.blog/changelog/2025-07-31-npm-trusted-publishing-with-oidc-is-generally-available/. Published 31 July 2025.

[19] GitHub Advisory Database. Embedded malware in ua-parser-js (GHSA-pjwm-rvh2-c87w, CVE-2021-4229), October 2021. URL https://github.com/advisories/GHSA-pjwm-rvh2-c87w. Published 22 October 2021.

[20] Petter Holme, Beom Jun Kim, Chang No Yoon, and Seung Kee Han. Attack vulnerability of complex networks. *Physical Review E*, 65(5):056109, 2002. doi: 10.1103/PhysRevE.65.056109.

[21] Jeffrey O. Kephart and Steve R. White. Directed-graph epidemiological models of computer viruses. In *Proceedings of the 1991 IEEE Computer Society Symposium on Research in Security and Privacy*, pages 343–359. IEEE, 1991. doi: 10.1109/RISP.1991.130801.

[22] William Ogilvy Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927. doi: 10.1098/rspa.1927.0118.

[23] Riivo Kikas, Georgios Gousios, Marlon Dumas, and Dietmar Pfahl. Structure and evolution of package dependency networks. In *Proceedings of the 14th IEEE/ACM International Conference on Mining Software Repositories (MSR)*, pages 102–112. IEEE, 2017. doi: 10.1109/MSR.2017.55.

[24] Ashish Kurmi. Shai-hulud: Self-replicating worm compromises 500+ NPM packages. StepSecurity Blog, September 2025. URL https://www.stepsecurity.io/blog/ctrl-tinycolor-and-40-npm-packages-compromised. Published 15 September 2025.

[25] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. SoK: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1509–1526. IEEE, 2023. doi: 10.1109/SP46215.2023.10179304.

[26] LavaMoat contributors. @lavamoat/allow-scripts. GitHub repository. URL https://github.com/LavaMoat/LavaMoat/tree/main/packages/allow-scripts.

[27] Cong Li, Huijuan Wang, and Piet Van Mieghem. Epidemic threshold in directed networks. *Physical Review E*, 88(6):062802, 2013. doi: 10.1103/PhysRevE.88.062802.

[28] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 672–684. ACM, 2022. doi: 10.1145/3510003.3510142.

[29] Sergei Maslov and Kim Sneppen. Specificity and stability in topology of protein networks. *Science*, 296(5569):910–913, 2002. doi: 10.1126/science.1065103.

[30] Amir M. Mir, Mehdi Keshani, and Sebastian Proksch. On the effect of transitivity and granularity on vulnerability propagation in the Maven ecosystem. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 201–211. IEEE, 2023. doi: 10.1109/SANER56733.2023.00028.

[31] M. E. J. Newman. Spread of epidemic disease on networks. *Physical Review E*, 66(1):016128, 2002. doi: 10.1103/PhysRevE.66.016128.

[32] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367. ACM, 2022. doi: 10.1145/3548606.3560596.

[33] npm, Inc. npm config: ignore-scripts. npm Docs (CLI v11). URL `https://docs.npmjs.com/cli/v11/using-npm/config#ignore-scripts`.

[34] npm, Inc. kik, left-pad, and npm. The npm Blog, March 2016. URL `https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm`. Published 23 March 2016.

[35] npm, Inc. Details about the event-stream incident. The npm Blog, November 2018. URL `https://blog.npmjs.org/post/180565383195/details-about-the-event-stream-incident`. Published 27 November 2018.

[36] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber's knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, Lecture Notes in Computer Science, pages 23–43. Springer, 2020. doi: 10.1007/978-3-030-52683-2_2.

[37] Kush Pandya, Peter van der Zee, Olivia Brown, and Socket Research Team. Updated and ongoing supply chain attack targets CrowdStrike npm packages. Socket Blog, September 2025. URL `https://socket.dev/blog/ongoing-supply-chain-attack-targets-crowdstrike-npm-packages`. Published 16 September 2025.

[38] Romualdo Pastor-Satorras and Alessandro Vespignani. Epidemic spreading in scale-free networks. *Physical Review Letters*, 86(14):3200–3203, 2001. doi: 10.1103/PhysRevLett.86.3200.

[39] Romualdo Pastor-Satorras and Alessandro Vespignani. Immunization of complex networks. *Physical Review E*, 65(3):036104, 2002. doi: 10.1103/PhysRevE.65.036104.

[40] Romualdo Pastor-Satorras, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani. Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3):925–979, 2015. doi: 10.1103/RevModPhys.87.925.

[41] Donald Pinckney, Federico Cassano, Arjun Guha, and Jonathan Bell. A large scale analysis of semantic versioning in NPM. In *Proceedings of the 20th IEEE/ACM International Conference*

on Mining Software Repositories (MSR), pages 485–497. IEEE, 2023. doi: 10.1109/MSR59073.2023.00073.

[42] pnpm contributors. pnpm v10.0.0 release notes. GitHub release, January 2025. URL `https://github.com/pnpm/pnpm/releases/tag/v10.0.0`. Released 7 January 2025.

[43] Tom Preston-Werner. Semantic versioning 2.0.0, 2013. URL `https://semver.org/spec/v2.0.0.html`.

[44] Hila Ramati, Merav Bar, Gal Benmocha, and Gili Tikochinski. Shai-hulud 2.0 supply chain attack: 25k+ repos exposing secrets. Wiz Blog, November 2025. URL `https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack`. Published 24 November 2025.

[45] Xavier René-Corail. Our plan for a more secure npm supply chain. The GitHub Blog, September 2025. URL `https://github.blog/security/supply-chain-security/our-plan-for-a-more-secure-npm-supply-chain/`. Published 22 September 2025.

[46] Michael Robinson, Sajal Halder, Muhammad Ejaz Ahmed, Muhammad Ikram, Seyit Camtepe, and Hyoungshick Kim. Original sin of npm: A study on vulnerability propagation in JavaScript dependency networks, 2026. URL `https://arxiv.org/abs/2604.17668`. Accepted at ACM AsiaCCS 2026 per the arXiv listing.

[47] Giulio Rossetti, Letizia Milli, Salvatore Rinzivillo, Alina Sîrbu, Dino Pedreschi, and Fosca Giannotti. NDlib: A python library to model and analyze diffusion processes over complex networks. *International Journal of Data Science and Analytics*, 5(1):61–79, 2018. doi: 10.1007/s41060-017-0086-6.

[48] Christian M. Schneider, Tamara Mihaljev, Shlomo Havlin, and Hans J. Herrmann. Suppressing epidemics with a limited amount of immunization units. *Physical Review E*, 84(6):061911, 2011. doi: 10.1103/PhysRevE.84.061911.

[49] William Schueller and Johannes Wachs. Modeling interconnected social and technical risks in open source software ecosystems. *Collective Intelligence*, 3(1), 2024. doi: 10.1177/26339137241231912.

[50] SLSA Community. SLSA specification, version 1.0. Open Source Security Foundation / Linux Foundation, 2023. URL `https://slsa.dev/spec/v1.0/`.

[51] Christophe Tafani-Dereeper and Sebastian Obregoso. The shai-hulud 2.0 npm worm: analysis, and what you need to know. Datadog Security Labs, November 2025. URL `https://securitylabs.datadoghq.com/articles/shai-hulud-2.0-npm-worm/`. Published 25 November 2025, updated 4 December 2025.

[52] Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. Defending against package typosquatting. In *Network and System Security (NSS)*, Lecture Notes in Computer Science, pages 112–131. Springer, 2020. doi: 10.1007/978-3-030-65745-1_7.

[53] Unit 42, Palo Alto Networks. "shai-hulud" worm compromises npm ecosystem in supply chain attack. Unit 42 Threat Research, 2025. URL `https://unit42.paloaltonetworks.com/npm-supply-chain-attack/`. First published September 2025; updated 25–26 November 2025 and 3 December 2025.

[54] Unit 42, Palo Alto Networks. The npm threat landscape: Attack surface and mitigations. Unit 42 Threat Research, 2026. URL https://unit42.paloaltonetworks.com/monitoring-npm-supply-chain-attacks/. Updated 15 July 2026.

[55] Yang Wang, Deepayan Chakrabarti, Chenxi Wang, and Christos Faloutsos. Epidemic spreading in real networks: An eigenvalue viewpoint. In *Proceedings of the 22nd International Symposium on Reliable Distributed Systems (SRDS)*, pages 25–34. IEEE, 2003. doi: 10.1109/RELDIS.2003.1238052.

[56] Elizabeth Wyss, Alexander Wittman, Drew Davidson, and Lorenzo De Carli. Wolf at the door: Preventing install-time attacks in npm with latch. In *Proceedings of the 2022 ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, pages 1139–1153. ACM, 2022. doi: 10.1145/3488932.3523262.

[57] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. What are weak links in the npm supply chain? In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 331–340. ACM, 2022. doi: 10.1145/3510457.3513044.

[58] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. On the impact of security vulnerabilities in the npm and RubyGems dependency networks. *Empirical Software Engineering*, 27(5), 2022. doi: 10.1007/s10664-022-10154-1.

[59] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small world with high risks: A study of security threats in the npm ecosystem. In *Proceedings of the 28th USENIX Security Symposium*, pages 995–1010. USENIX Association, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman.

[60] Cliff Changchun Zou, Weibo Gong, and Don Towsley. Code red worm propagation modeling and analysis. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, pages 138–147. ACM, 2002. doi: 10.1145/586110.586130.